

Adversarial Attacks on NLP Models

Progress Report 1

Ignacio Bayón Jiménez-Ugarte
202105547@alu.comillas.edu

Jorge Gómez Azor
202107216@alu.comillas.edu

Adrián López-Lanchares Echezarreta
202107370@alu.comillas.edu

Carlos Mazuecos Refillo
202106145@alu.comillas.edu

12th April 2024

1 Introduction

Throughout this week we have been familiarizing ourselves with various models within Adversarial Attacks. These can be divided into Black-Box and White-Box Attacks depending on the access we have to the model. In Black-Box Attacks, we have very limited access to the model, as we only know its inputs and outputs. Therefore, the only approach possible is to alter the inputs in order to (fool?) the network. On the other hand, in White-Box attacks, we have complete access to the model: architecture, parameters, activation function, loss function, input and output. That grants us access to its inner calculations. [Zha+19]

Our work is focused on three different Black-Box attacks, poisoning, synonym and semantic, and a single White-Box attack, the gradient based attack. These will be tested with two different architectures of neural based models: the MultiLayer Perceptron (MLP) and the Long Short Term Memory (LSTM) network. The following tables show the results obtained up to date. The X's mark the areas we have not yet covered.

Models	MLP				LSTM			
Accuracies	Clean	Poisoning	Synonyms	Semantic	Clean	Poisoning	Synonyms	Semantic
Sentiment An. I	X	X	X	X	X	X	X	X
Sentiment An. II	X	X	X	X	X	X	X	X
Sentence Classific.	X	X	X	X	X	X	X	X

Table 1: Black Box Attacks

Models	MLP		LSTM	
Accuracies	Clean	Gradient Based	Clean	Gradient Based
Sentiment An. I	X	X	X	X
Sentiment An. II	X	X	X	X
Sentence Classific.	X	X	X	X

Table 2: White Box Attacks

2 Adversarial Attacks

Up to date, we have been able to advance in poisoning and, mostly, in gradient based attacks. We have already started to implement the gradient descent attack [Guo+21]. Besides this attack, we want to fully implement, as

specified in the tables above, poisoning and syntactic based attack. We have already read research on them but have not managed to develop any usable code. They will grant us the opportunity to contrast how the different attacks can affect the performance of the models mentioned.

3 Implementation

We have created our own models for classification and sentiment analysis. Accuracies of 0.8 for sentiment analysis and 0.85 in multi-classification text with the use of a RNN (LSTM). We used 2 different datasets in order to see the effects of the different text attacks in different models. We trained the models using a Sentiment Analysis dataset and a Multiclass Classification dataset. We used the same preprocessing for both datasets, with the distinction of a one-hot encoded label tensor for the Multiclass Classification dataset. The input texts were transformed into vectors of integers, which were then passed through an Embedding layer in both models. Both models behave in a very similar way. After passing the inputs through the embedding layer, the data goes through an LSTM layer. Finally, there is a dropout layer, a linear layer mapping the data to the desired output size (1 for sentiment analysis, 4 for multiclass classification), and a Sigmoid / Softmax layer turns the outputs into probabilities.

In addition, we have started to implement a gradient descent adversarial attack:

```
def generate_gdba(model, input_sequence, true_label, epsilon=0.1, iterations=10, perturbation=0.1):
    input_sequence_var = torch.tensor(input_sequence, requires_grad=True, dtype=torch.float)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD([input_sequence_var], lr=epsilon)

    for i in range(iterations):
        optimizer.zero_grad()
        output = model(input_sequence_var.unsqueeze(0))
        loss = criterion(output, torch.tensor([true_label]))
        loss.backward()

        # Restricted perturbation
        perturbation = epsilon * input_sequence_var.grad.sign()
        perturbation = torch.clamp(perturbation, -perturbation_limit, perturbation_limit)

        input_sequence_var.data = input_sequence_var.data + perturbation
        input_sequence_var.data = torch.clamp(input_sequence_var.data, 0, 1)

    return input_sequence_var.detach().numpy()
```

4 Next Steps

We are currently centered on finishing the different attacks and fill in the table. We would be able then to get our own conclusions of the ability these state-of-the-art techniques have of attacking different models. On one hand, the MLP is a sufficiently simple model to see effects without needing excessively complex techniques. On the other hand, the LSTM will be able to test the ability of our more advanced technique, the gradient based adversarial attack.

References

- [Zha+19] W. E. Zhang et al. “Adversarial Attacks on Deep Learning Models in Natural Language Processing: A Survey”. In: (2019), p. 40. URL: <https://arxiv.org/pdf/1901.06796.pdf>.
- [Guo+21] C. Guo et al. “Gradient-based Adversarial Attacks against Text Transformers”. In: (2021), p. 11. URL: <https://arxiv.org/pdf/2104.13733.pdf>.